

Workshop on

Filippov-ODEs with State-dependent Switches

Friday, July 18, 2025 • CIP Pool 1 • 3rd Floor • Mathematikon

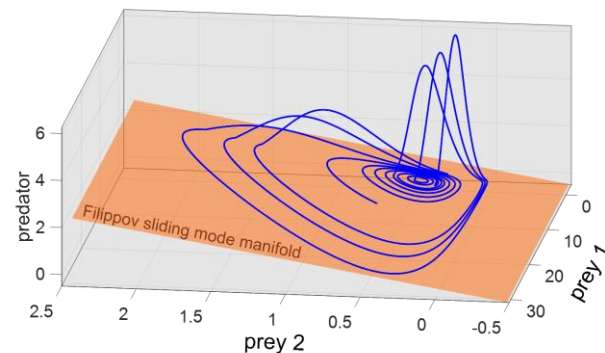
Course program

09:00 – 11:00 Introduction to Switched ODEs

- A canonical example
- A glimpse on the theory of switched ODEs: Switching functions
- Sensitivities: Derivatives w.r.t. to initial value and parameters
- Filippov sliding mode

11:00 – 12:00 Hands-On Practical: Part 1

- Short recap: Solving ODE in Matlab
- Applying IFDIFF to existing Matlab code



12:00 – 13:00 Lunch break (with finger food and beverages)

13:00 – 16:00 Hands-On Practical: Part 2

- Solving switched ODE problems with IFDIFF
- Formulating switched ODE systems and state-jumps
- Guided examples
- Sensitivity computation and usage in parameter estimation
- Diagnosing and avoiding common mistakes



WORKSHOP ON

FILIPPOV-ODEs WITH STATE-DEPENDENT SWITCHES

THEORY AND HANDS-ON PRACTICAL WITH IFDIFF

Andreas Sommer

Interdisciplinary Center for Scientific Computing (IWR)
Heidelberg University

andreas.sommer@iwr.uni-heidelberg.de

CO-ORGANIZERS

Marvin Lu Hertweck

Lev Gromov

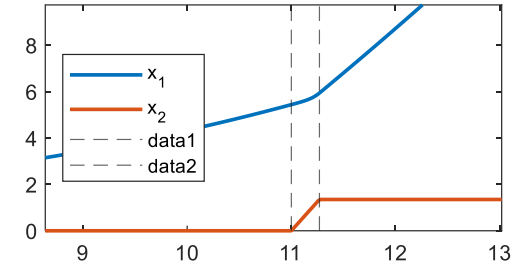
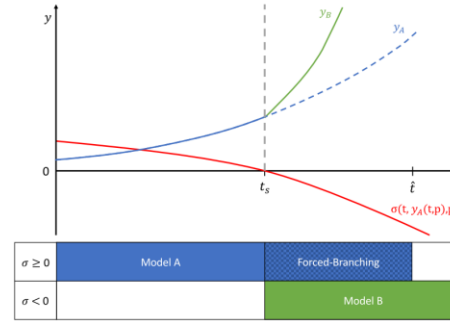
Pilar Keller

VECTOR 
STIFTUNG

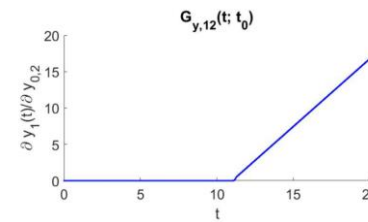
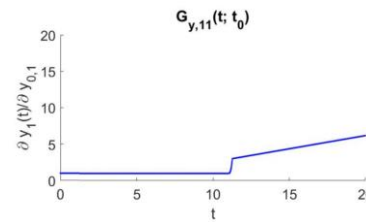
IFDIFF is financially supported by
Vector Stiftung (Project PERFDiff)

Outline

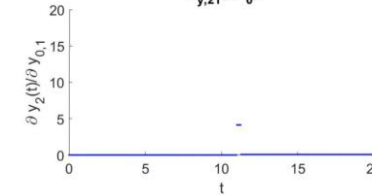
- ODEs with implicit (state-dependent)
 - A Canonical Example
 - A Glimpse on Switching Function Theory
 - Determination of Switching Points



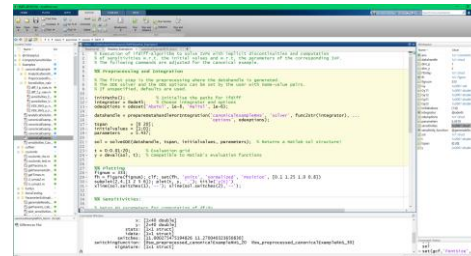
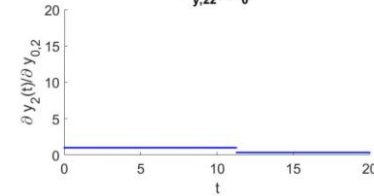
- Sensitivity Generation
 - Forward sensitivities
 - Propagation across switches



$G_{y,21}(t; t_0)$



$G_{y,22}(t; t_0)$

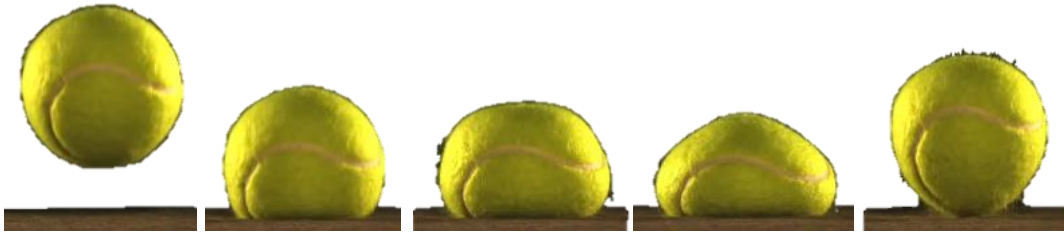


Motivation of Implicitly Switched ODEs

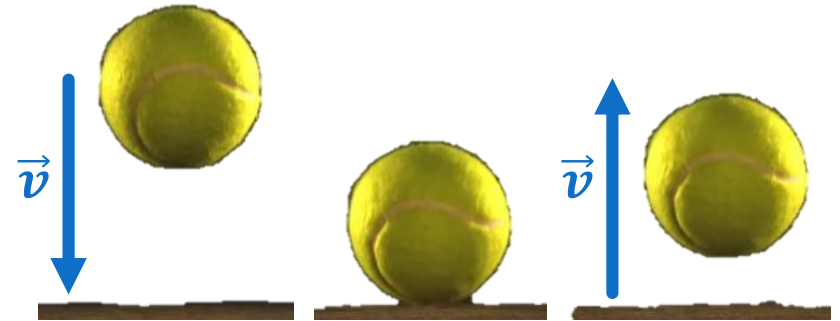
Switched Systems

- Switched systems often arise as simplifications of complicated continuous processes
- Example: bouncing ball

Modelling and simulation of complex material deformation is **very complicated**



Simplification: change in the velocity vector at the contact point



Switching event/model change
when the ball hits the ground

Motivation

- Setting: ODE/IVP $\dot{x} = f(t, x, p)$ with rhs function f coded as Matlab file `rhs_f.m`
- **Rapid prototyping** – during model development or model extension
 - quick testing of ideas

state-dependent model switching

```
function dx = rhs_f(t,x,p)
...
if x(2) > vsonic
    dx(3) = submodel_supersonic(t,x,p)
else
    dx(3) = submodel_subsonic(t,x,p);
end
...
end
```

nonnegativity enforcement:

```
function dx = rhs_f(t,x,p)
...
k1 = x(2)*x(3) - x(4);
⇒ k1 = max(0, k1);
dx(6) = x(3) * x(4) * sqrt(k1)
...
end
```

- Introduced non-differentiabilities require proper treatment $\Rightarrow$ End of rapid prototyping ☹
- **Goal:** Remove mathematical workload during model development

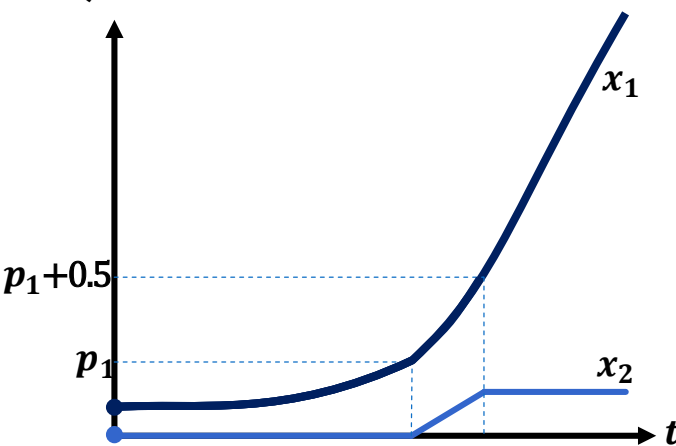
Switched ODEs. By Example.

Canonical Example

- 2-dimensional system with state-dependent switching:

$$\begin{aligned}\dot{x}(t) &= f(t, x, p) \\ \dot{x}_1(t) &= 0.01 \cdot t^2 + x_2(t)^3 \\ \dot{x}_2(t) &= \begin{cases} 0 & \text{if } x_1(t) < p_1 \\ 5 & \text{if } p_1 \leq x_1(t) < p_1 + 0.5 \\ 0 & \text{if } x_1(t) \geq p_1 + 0.5 \end{cases} \\ x(0) &= (1, 0)^T \quad p_1 = 5.437\end{aligned}$$

- Qualitative sketch of solution:



Qualitative solution construction

- System is monotonically increasing

- Start at $(x_1, x_2) = (1, 0)$.

$\dot{x}_2 = 0$, since $x_1 < p_1$ for a while

→ 2nd component $x_2 \equiv 0$ for a while

cubic part $x_2(t)^3$ in 1st component is 0

→ 1st component $x_1(t)$ is a cubic polynomial in t

- After a while, $x_1 \geq p_1$

→ 2nd component rises linearly with slope 5

→ 1st component rises even quicker now due to $x_2(t)^3$

- After a while, $x_1 \geq p_1 + 0.5$

→ 2nd component stays constant (zero slope)

→ 1st component keeps rising

Let's try in Matlab!

How switched ODEs are frequently “solved”...



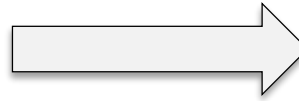
Just ignore the switches and integrate!
The error control will do fine!

No. It won't.

Malicious Example: Unnoticed Disaster

- 2-dimensional system with state-dependent switching:

$$\begin{aligned}\dot{x}(t) &= f(t, x, p) \\ \dot{x}_1(t) &= 0.01 \cdot t^2 + x_2(t)^3 \\ \dot{x}_2(t) &= \begin{cases} 0 & \text{if } x_1(t) < p_1 \\ 5 & \text{if } p_1 \leq x_1(t) < p_1 + 0.5 \\ 0 & \text{if } x_1(t) \geq p_1 + 0.5 \end{cases} \\ x(0) &= (1, 0)^T \quad p_1 = 5.437\end{aligned}$$



```
function dx = f(t,x,p)
```

```
    dx = zeros(2,1);  
    dx(1) = 0.01 * t.^2 + x(2).^3;
```

```
    if x(1) < p(1)  
        dx(2) = 0;  
    else  
        if x(1) < p(1) + 0.5  
            dx(2) = 5;  
        else  
            dx(2) = 0;  
        end  
    end
```

```
end
```

```
% problem setup
```

```
tspan = [0 20];
```

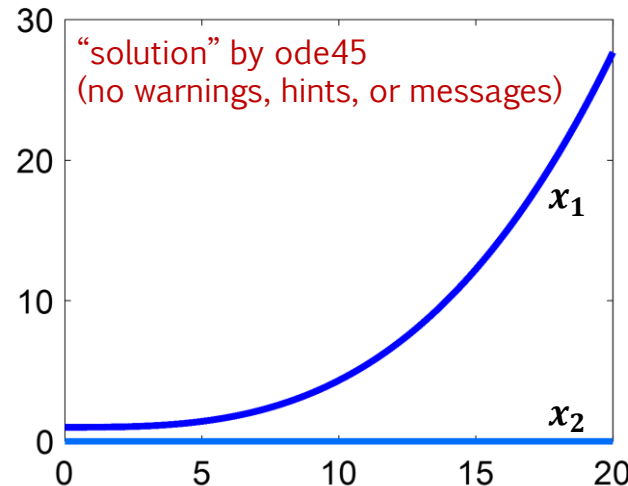
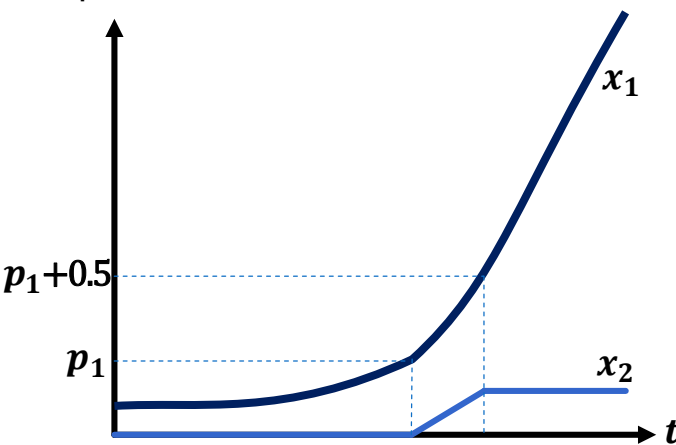
```
x0 = [1 0];
```

```
p = 5.437;
```

```
% integration
```

```
sol = ode45(@(t,x) f(t,x,p), tspan, x0);
```

- qualitative sketch of solution:



Existence and Uniqueness of ODE Solutions

- ODE – Ordinary Differential Equation

→ Evolution of a system depending on its state.

$$\dot{x}(t) = f(t, x(t)), \quad t \in [t_0, t_f]$$

- IVP – Initial Value Problem

→ A value x_0 at the initial time point t_0 is prescribed: $x(t_0) = x_0$

PICARD-LINDELÖF or CAUCHY-LIPSCHITZ Theorem

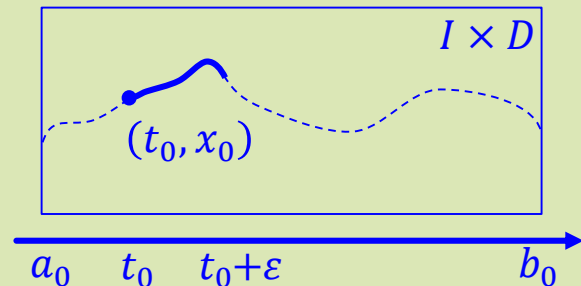
Let $I := [a_0, b_0]$ a (time) interval

Let $D := [a_1, b_1] \times [a_2, b_2] \times \cdots \times [a_n, b_n] \subseteq \mathbb{R}^n$ be a closed rectangle.

Let $(t_0, x_0) \in \text{int}(I \times D)$ be a point in the interior.

Let $f: I \times D \rightarrow \mathbb{R}^n$, $(t, x) \mapsto f(t, x)$ be continuous in t and **locally LIPSCHITZ-continuous**^(★) in x (on D).

Then $\exists \varepsilon > 0$, such that the IVP $\dot{x}(t) = f(t, x(t))$, $x(t_0) = x_0$ has a unique solution on $[t_0, t_0 + \varepsilon]$



If f is **globally LIPSCHITZ-continuous** on D , then the solution is globally unique.

(★) Function f is LIPSCHITZ-continuous with constant $L > 0$, if $|f(x_2) - f(x_1)| \leq L \cdot |x_2 - x_1|$

Piecewise Solutions

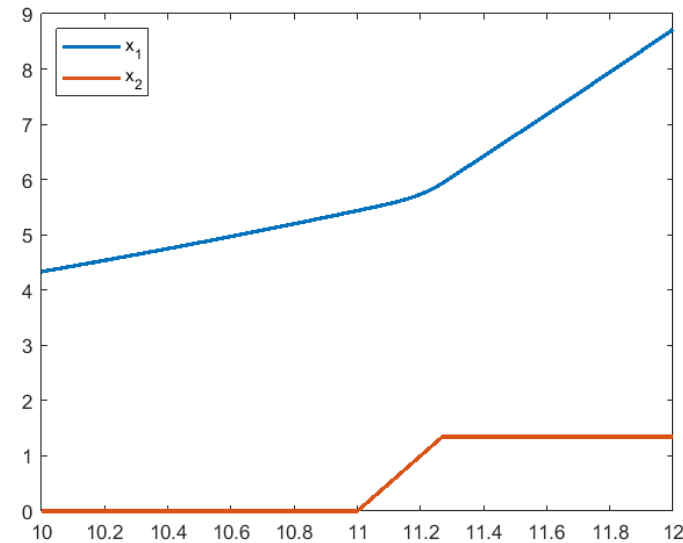
- Reconsider the canonical example from the beginning
- RHS function f is a **concatenation of piecewise LIPSCHITZ** functions
→ Solution $x(t)$ is piecewise continuously differentiable
- If there are **finitely many** isolated “jumps” in the RHS function f
the existence and uniqueness results hold for each interval solution
- We can **construct a complete solution** by **stitching the interval solutions** (carefully)
- CAVEAT: We usually don't know the switching time points!
→ Details in the second session

$$\dot{x}(t) = f(t, x, p)$$

$$\dot{x}_1(t) = 0.01 \cdot t^2 + x_2(t)^3$$

$$\dot{x}_2(t) = \begin{cases} 0 & \text{if } x_1(t) < p_1 \\ 5 & \text{if } p_1 \leq x_1(t) < p_1 + 0.5 \\ 0 & \text{if } x_1(t) \geq p_1 + 0.5 \end{cases}$$

$$x(0) = (1, 0)^T \quad p_1 = 5.437$$



Numerical Solution of ODE-IVPs

Quadrature-based Integration Methods

- The solution of autonomous ODE-IPV

$$\dot{x} = f(x), \quad x(t_0) = x_0$$

can be (formally) written as integral:

$$x(t) = x_0 + \int_{t_0}^t f(x(s)) \, ds$$

- **Idea:** Use a **quadrature rule** to approximate the integral.
- Current approximation $x^{(k)}$ at time t_k , move forward by time step Δt :

$$x^{(k+1)} := x^{(k)} + \int_{t_k}^{t_k + \Delta t} f(x(s)) \, ds$$

Integration Methods

- There is a plethora of integration methods for ODEs!
- One-step methods: The next value $x^{(k+1)}$ only depends on previous approximation $x^{(k)}$
- Multi-step methods: The next value $x^{(k+1)}$ depends on multiple approximations $x^{(k)}, x^{(k-1)}, x^{(k-2)}, \dots$
- Explicit Methods: The next value $x^{(k+1)}$ can be directly computed
- Implicit Methods: The next value $x^{(k+1)}$ requires the solution of a nonlinear equation system
Implicit methods are (usually) more suitable for stiff problems
- Other methods based on fundamentally different approaches
(e.g. multi-derivative approaches, quantized state systems method, ...)

One-Step Methods

$$x^{(k+1)} := x^{(k)} + \int_{t_k}^{t_k + \Delta t} f(x(s)) \, ds$$

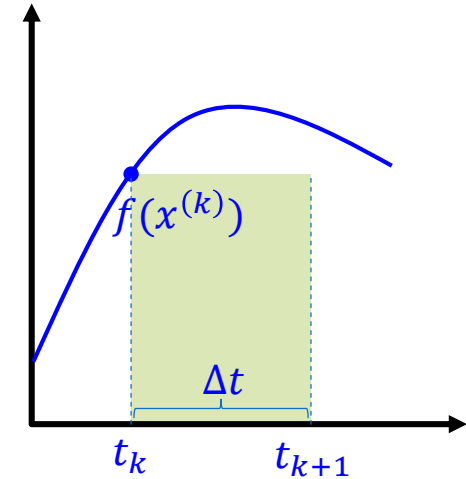
- Most simple:

Use $f(x^{(k)})$ as constant approximation of $f(x(s))$ in interval $[t_k, t_k + \Delta t]$

$$x^{(k+1)} := x^{(k)} + \Delta t \cdot f(x^{(k)})$$

We can compute the next iterate $x^{(k+1)}$ immediately

→ **Explicit Euler**

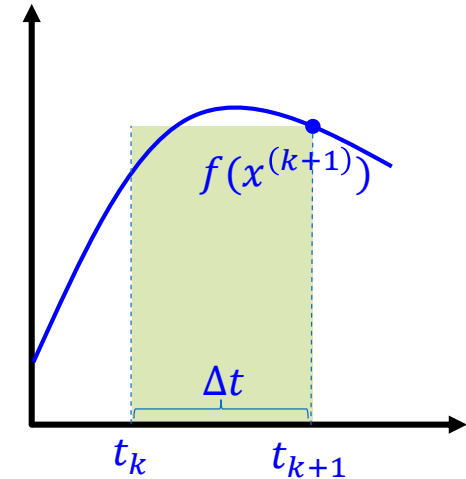


- We could also use the value at the interval end as constant approximation

$$x^{(k+1)} := x^{(k)} + \Delta t \cdot f(x^{(k+1)})$$

Have to solve a nonlinear equation to compute $x^{(k+1)}$

→ **Implicit Euler**



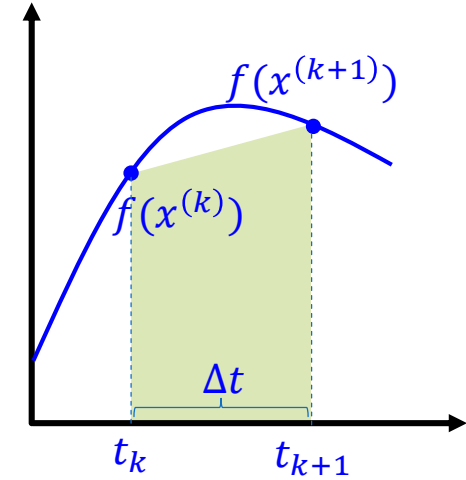
One-Step Methods

$$x^{(k+1)} := x^{(k)} + \int_{t_k}^{t_k + \Delta t} f(x(s)) \, ds$$

- We can also evaluate at both ends

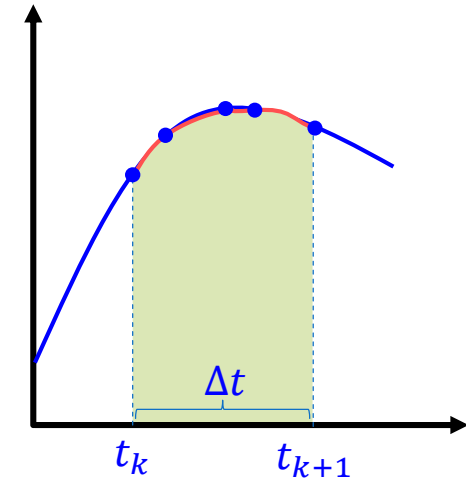
$$x^{(k+1)} := x^{(k)} + \frac{\Delta t}{2} \cdot (f(x^{(k)}) + f(x^{(k+1)}))$$

→ Trapezoidal method (implicit)



- We can also use several other points in $[t_k, t_{k+1}]$ and use more sophisticated quadrature formulas

→ RUNGE-KUTTA methods (explicit and implicit)

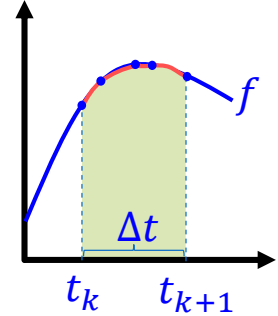


RUNGE-KUTTA Methods

- Huge class of one-step methods for ODE-IVPs: $\dot{x}(t) = f(t, x(t)), \quad x(t_0) = x_0$

- RK method with s stages:
$$x^{(k+1)} := x^{(k)} + \Delta t \cdot \sum_{j=1}^s b_j k_j$$

 b_j : quadrature weights
 k_j : evaluations at intermediate points



- Method coefficients:
$$k_j = f\left(t_k + \Delta t \cdot c_j, \quad x^{(k)} + \Delta t \sum_{l=1}^s a_{jl} k_l\right) \quad (j = 1, \dots, s)$$

 c_j : knots (evaluation times)
 a_{jl} : quadrature weights for k_l

- Shorthand notation:
BUTCHER tableau

$$\begin{array}{c|c} \mathbf{c} & \mathbf{A} \\ \hline & \mathbf{b}^T \end{array} = \begin{array}{c|ccccc} c_1 & a_{11} & a_{12} & \dots & a_{1s} \\ c_2 & a_{21} & a_{22} & \dots & a_{2s} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_s & a_{s1} & a_{s2} & \dots & a_{ss} \\ \hline & b_1 & b_2 & \dots & b_s \end{array}$$

Note:

$a_{jl} = 0 \quad \forall l \geq j$ (A is lower triangular)
 $\Rightarrow$ explicit method

Explicit Euler:

$$x^{(k+1)} := x^{(k)} + \Delta t \cdot f(t_k, x^{(k)})$$

$$\begin{array}{r|rr} s = 1 & 0 & 0 \\ a_{11} = 0, b_1 = 1, c_1 = 0 & \hline & 1 \end{array}$$

Implicit Euler:

$$x^{(k+1)} := x^{(k)} + \Delta t \cdot f(t_{k+1}, x^{(k+1)})$$

$$\begin{array}{r|rr} s = 1 & 1 & 1 \\ a_{11} = 1, b_1 = 1, c_1 = 1 & \hline & 1 \end{array}$$

Step Size Strategies

- Constant step size Δt is usually inefficient
- Idea: Choose step size Δt_k such that **error is controlled**
- Local discretization error: $e^{(k)} := x(t_k) - x^{(k)}$ NOTE: True value $x(t_k)$ is not available!
- Use a “more accurate” numerical approximation $\tilde{x}^{(k)}$ instead of true value $x(t_k)$

$$x^{(k+1)} := x^{(k)} + \Delta t_k \cdot \sum_{j=1}^s b_j k_j$$

approximation of order p

$$\tilde{x}^{(k+1)} := x^{(k)} + \Delta t_k \cdot \sum_{j=1}^s \tilde{b}_j k_j$$

approximation of order $p + 1$

c_1	a_{11}	a_{12}	$\dots$	a_{1s}
c_2	a_{21}	a_{22}	$\dots$	a_{2s}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_s	a_{s1}	a_{s2}	$\dots$	a_{ss}
	b_1	b_2	$\dots$	b_s
	$\tilde{b}_1$	$\tilde{b}_2$	$\dots$	$\tilde{b}_s$

BUTCHER tableau

→ Efficient: We can reuse the stage values k_j

- Use error approximation to adjust step size Δt_k

$$\tilde{e}^{(k+1)} := \tilde{x}^{(k+1)} - x^{(k+1)} = \Delta t_k \cdot \sum_{i=1}^s (\tilde{b}_i - b_i) k_i$$

Dormand-Prince: Matlab's ode45

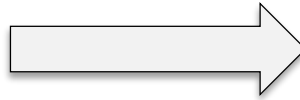
0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	-56/15	32/9				
8/9	19372/6561	-25360/2187	64448/6561	-212/729			
1	9017/3168	-355/33	46732/5247	49/176	-5103/18656		
1	35/384	0	500/1113	125/192	-2187/6784	11/84	
	35/384	0	500/1113	125/192	-2187/6784	11/84	0
	5179/57600	0	7571/16695	393/640	-92097/339200	187/2100	1/40

Back to the „Canonical Example“

Malicious Example: Unnoticed Disaster

- 2-dimensional system with state-dependent switching:

$$\begin{aligned}\dot{x}(t) &= f(t, x, p) \\ \dot{x}_1(t) &= 0.01 \cdot t^2 + x_2(t)^3 \\ \dot{x}_2(t) &= \begin{cases} 0 & \text{if } x_1(t) < p_1 \\ 5 & \text{if } p_1 \leq x_1(t) < p_1 + 0.5 \\ 0 & \text{if } x_1(t) \geq p_1 + 0.5 \end{cases} \\ x(0) &= (1, 0)^T \quad p_1 = 5.437\end{aligned}$$



```
function dx = f(t,x,p)
```

```
    dx = zeros(2,1);  
    dx(1) = 0.01 * t.^2 + x(2).^3;
```

```
    if x(1) < p(1)  
        dx(2) = 0;  
    else  
        if x(1) < p(1) + 0.5  
            dx(2) = 5;  
        else  
            dx(2) = 0;  
        end  
    end
```

```
end
```

```
% problem setup
```

```
tspan = [0 20];
```

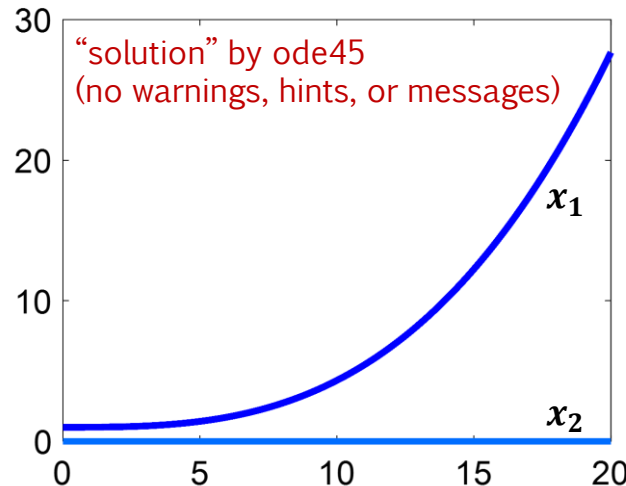
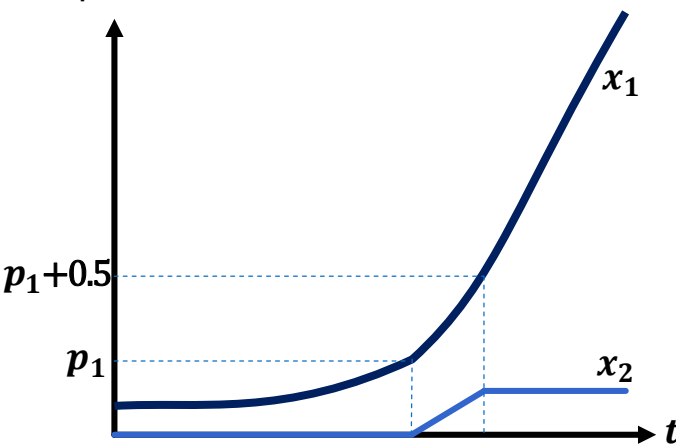
```
x0 = [1 0];
```

```
p = 5.437;
```

```
% integration
```

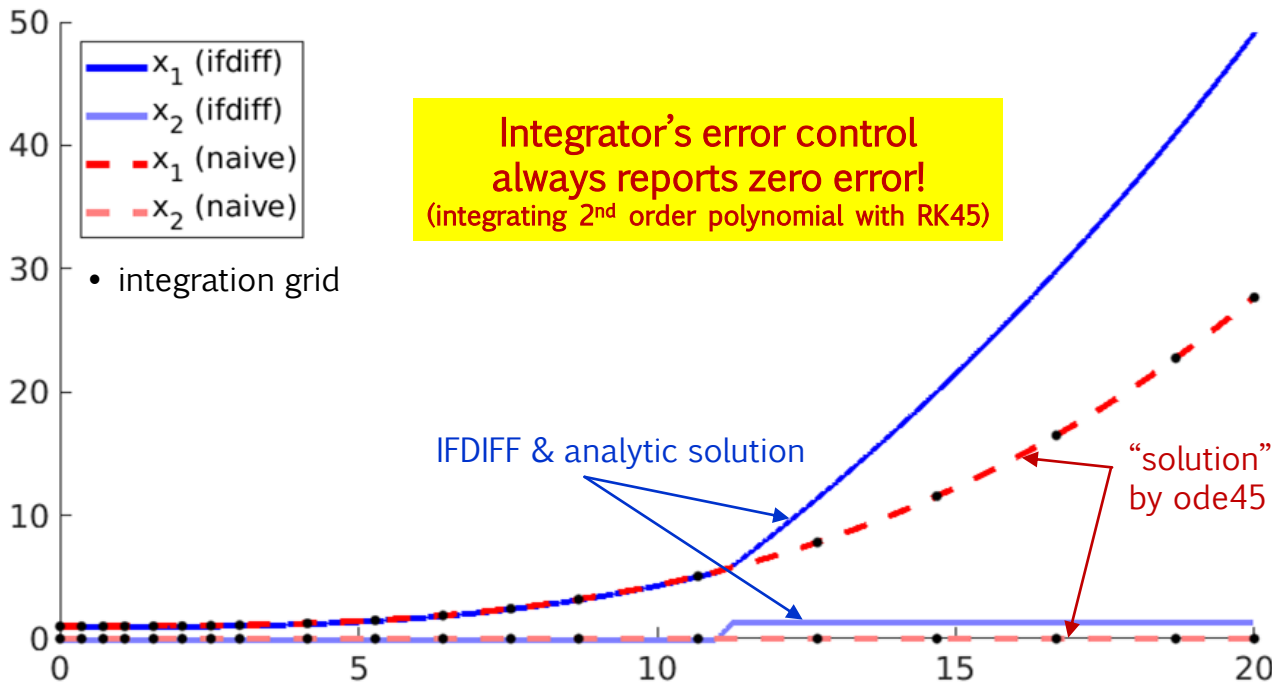
```
sol = ode45(@(t,x)f(t,x,p),tspan,x0);
```

- qualitative sketch of solution:



Malicious Example: Unnoticed Disaster

- naive integration (“rely on error control”) vs. IFDIFF
- program flow “jumps” from `dx(2)=0` to `dx(2)=0`



Matlab's ode45 (RK), independent of tolerances, no warnings/errors/hints

```
function dx = f(t,x,p)
```

```
dx = zeros(2,1);  
dx(1) = 0.01 * t.^2 + x(2).^3;
```

```
if x(1) < p(1)  
    dx(2) = 0;  
else  
    if x(1) < p(1) + 0.5  
        dx(2) = 5;  
    else  
        dx(2) = 0;  
    end  
end
```

```
end
```

```
% problem setup
```

```
tspan = [0 20];  
x0 = [1 0];  
p = 5.437;
```

```
% integration
```

```
sol = ode45(@(t,x)f(t,x,p),tspan,x0);
```

How switched ODEs should be integrated

Switched ODEs: A Glimpse onto Solution Theory

- Switched IVP: $\dot{x}(t) = f(t, x(t), p, \text{sign } \sigma(t, x(t), p))$ $x(t_0) = x_0(p)$
- Switching events t_s characterized by zero crossings: $\sigma(t_s, x, p) = 0$

- Technical assumptions:

- Individual “models” can be evaluated in an environment of each switch
- Isolated switching points

- Strategy:** Monitor σ during integration. Stop and restart at zero crossings.

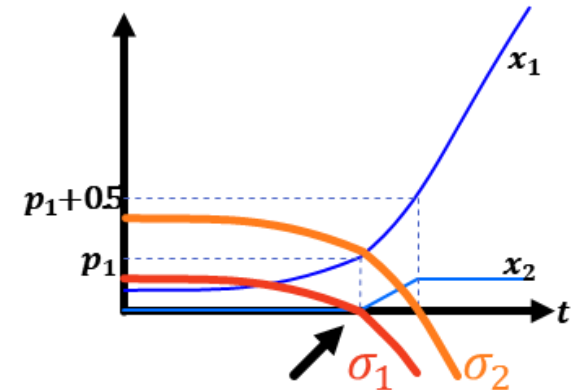
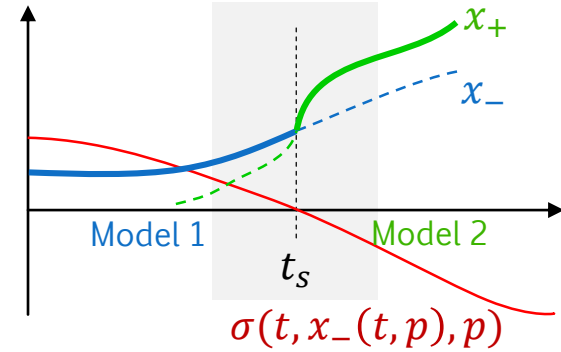
- Caveats:**
 - requires **extensive re-formulation** even for small model changes $\Rightarrow$ bye bye rapid prototyping ☹
 - n conditionals $\Rightarrow$ up to 2^n switching functions (impracticable for $n > 20$)

- Canonical Example:

$$\begin{aligned} \dot{x}_1(t) &= 0.01 \cdot t^2 + x_2(t)^3 \\ \dot{x}_2(t) &= \begin{cases} 0 & \text{if } x_1(t) < p_1 \\ 5 & \text{if } p_1 \leq x_1(t) < p_1 + 0.5 \\ 0 & \text{if } x_1(t) \geq p_1 + 0.5 \end{cases} \\ x(0) &= (1, 0)^T \quad p_1 = 5.437 \end{aligned}$$

Suitable switching functions

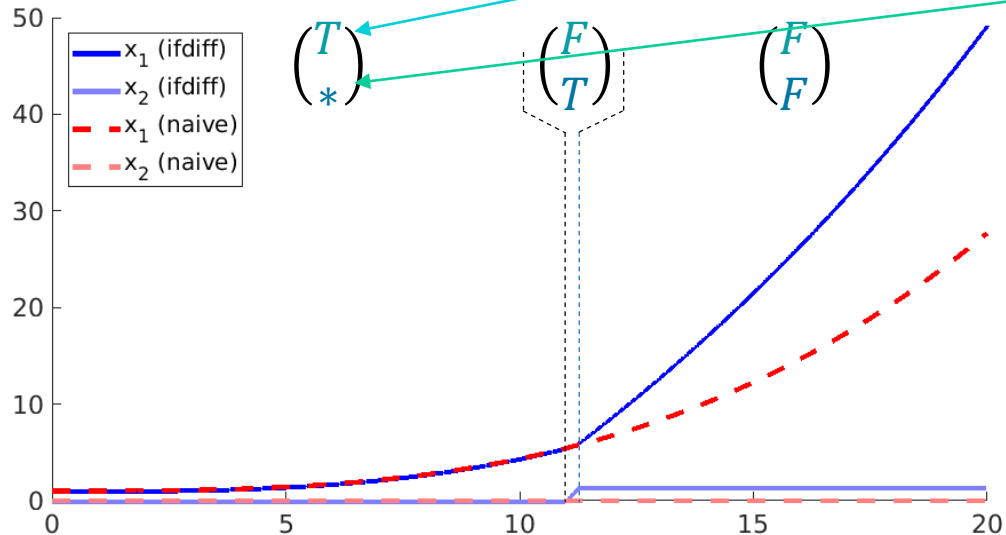
$$\begin{aligned} \sigma_1(t, x, p) &= p_1 - x_1(t) \\ \sigma_2(t, x, p) &= p_1 + 0.5 - x_1(t) \end{aligned}$$



Where is the Switching Function in a Matlab Code?

Surveilling Switches using Signatures and Augmented IFs

- Switching functions $\sigma(\cdot)$ implicitly given in the code:
 - e.g. as the condition in `if`-expressions
 - usually depend on intermediate values, sub-functions, etc.
- IFDIFF surveils the switching behavior
 - Store the **branching pattern (signature)** of individual non-differentiable operators and **monitor changes** in that signature



```
function dx = f(t,x,p)

dx = zeros(2,1);
dx(1) = 0.01 * t.^2 + x(2).^3;

if x(1) < p(1)
    dx(2) = 0;
else
    if x(1) < p(1) + 0.5
        dx(2) = 5;
    else
        dx(2) = 0;
    end
end

end
```

Automatic processing in IFDIFF:

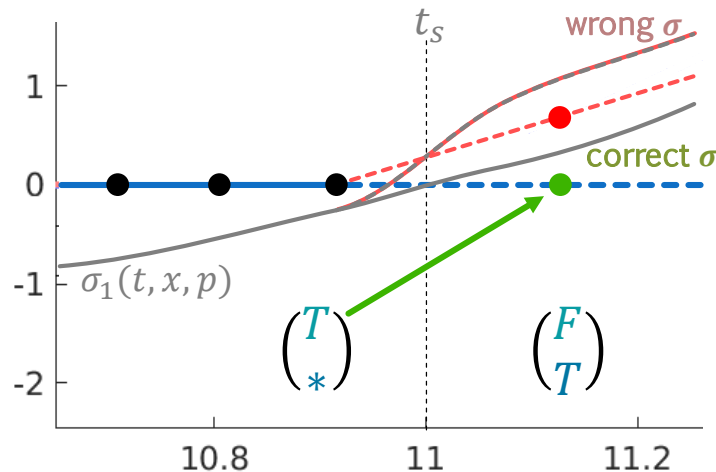
- augment `if`-expressions via source code transform* with monitoring and forced-branching capabilities

* based on Matlab's internal abstract syntax tree `mtree`

How to Compute the Switching Times?

Surveilling Switches using Signatures and Augmented IFs

- Determine root of switching function $\sigma(t, x, p)$ using continuous solution representation $x(t)$ from integrator
 - ⇒ free branching leads to wrong (mixed) model evaluation
 - ⇒ must not switch for consistency !!
- IFDIFF: Monitor the **if**-condition, detect switch, but force branching into previously active branch



- previous (accepted) integrator steps
- step with free branching (**wrong!**)
- step with forced branching (correct)

IFDIFF:

- Generates **only required** switching functions σ during integration
 - ⇒ breaks the curse of dimension
- Determines the root (switching time) up to machine precision

```
function dx = rhs_test(t,x,p)

    dx = zeros(2,1);
    dx(1) = 0.01 * t.^2 + x(2).^3;

    if x(1) < p(1)
        dx(2) = 0;
    else
        if x(1) < p(1) + 0.5
            dx(2) = 5;
        else
            dx(2) = 0;
        end
    end
end
```

Surveilling Switches using Signatures and Augmented IFs

- IFDIFF provides automatic preprocessing of RHS code files:

```
function dx = f(t,x,p)

dx = zeros(2,1);
dx(1) = 0.01 * t.^2 + x(2).^3;

if x(1) < p(1)
    dx(2) = 0;
else
    if x(1) < p(1) + 0.5
        dx(2) = 5;
    else
        dx(2) = 0;
    end
end
end
```

```
function dx = preprocessed_f(dhandle,t,x,p)
dx = zeros(2,1);
dx(1) = 0.01 * t.^2 + x(2).^3;
conditionValue = ctrlif( x(1) - (p(1)) >= 0, false, true, 1, 0, dhandle);
if conditionValue
    dx(2) = 0;
else
    conditionValue = ctrlif(x(1) - (p(1)+0.5) >= 0, false, true, 2, 0, dhandle);
    if conditionValue
        dx(2) = 5;
    else
        dx(2) = 0;
    end
end
end
```

```
function svalue = sw_rhs_preprocessed_f_10(dhandle,t,x,p)
    switchEval_if_1 = x( 1 ) - (p( 1 ));
    switching_value = switchEval_if_1;
end
```

switching function $\sigma_1(t,x,p)$

- Requires a suitable parser for Matlab programming language.
- Our implementation uses the best: Matlab's own parser.
Every Matlab program is internally represented by an "mtree" object, a tree-like structure representing the source.
(completely undocumented)

State Jumps

State Jumps

- Example: bouncing ball



- Simplification: **Swap sign** in the velocity vector upon ground contact
state jump

- Notation:

$$x_-(t_s) := \lim_{\substack{\varepsilon \rightarrow 0 \\ \varepsilon < 0}} x(t_s + \varepsilon)$$

$$x_+(t_s) := \lim_{\substack{\varepsilon \rightarrow 0 \\ \varepsilon > 0}} x(t_s + \varepsilon)$$

$$\Delta : I \times \mathbb{R}^n \rightarrow \mathbb{R}^n$$

$$(t, x_-(t)) \mapsto x_+(t)$$

jump function Δ

$$\vec{v}_+(t) := \Delta(t, \vec{v}_-(t)) = \vec{v}_-(t) \circ \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

Hadamard product

$$\vec{v}_-(t_s) = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \Rightarrow \begin{pmatrix} -1 \\ 0 \end{pmatrix} = \vec{v}_+(t_s)$$

- Frequently, the jump function also depends on system parameters: $\Delta = \Delta(t, x, p)$

State and Parameter Sensitivities (without Switches)

Forward Sensitivities of ODEs (without switches)

- Parametrized initial value problem: $\dot{x}(t) = f(t, x(t), p), \quad x(t_0) = x_0$ Solution: $x(t; t_0, x_0, p)$
- State and parameter sensitivities: $G_{x_0}(t) := \frac{dx}{dx_0}(t; t_0, x_0, p)$ and $G_p(t) := \frac{dx}{dp}(t; t_0, x_0, p)$
- Variational Differential Equations (VDE): $\dot{G}_{x_0}(t) = \frac{df}{dx}(t, x(t), p) \cdot G_{x_0}(t) \quad G_{x_0}(t_0) = \mathbf{1}$
- How (rather not) to compute:
 - FD / External Numerical Differentiation (END): $\frac{dx}{dx_{0,i}}(t) \approx \frac{x(t; t_0, x_0, p) - \overset{\text{different functions due to adaptivity}}{\hat{x}(t; t_0, x_0 + he_i, p)}}{h}$
very problematic, due to adaptive components
 - AD tools on whole integration scheme: problematic also due to adaptive components in integrator
- How to compute
 - Internal Numerical Differentiation (IND): freezing adaptive components (requires adjustment to integrator)
 - Solving VDE with numerical integrator: error controlled (requires continuous representation of solution y)

Sensitivity Propagation across Switches

Propagation of Sensitivities across Switches

▪ IVP: $\dot{x}(t) = f(t, x(t), p)$, $x(t_0) = x_0$ with solution: $x(t; t_0, x_0, p)$

▪ State sensitivities: $G_{x_0}(t) := \frac{dx}{dx_0}(t; t_0, x_0, p)$

▪ Integrator generates solution:

$$x_n = \left\{ \begin{array}{c} \text{integration} \\ \text{from } t_s \text{ to } t_1 \end{array} \right\} \circ \left\{ \begin{array}{c} \text{switch handling at } t_s \\ x_-(t_s) \mapsto x_+(t_s) \end{array} \right\} \circ \left\{ \begin{array}{c} \text{integration} \\ \text{from } t_0 \text{ to } t_s \end{array} \right\} (x_0)$$

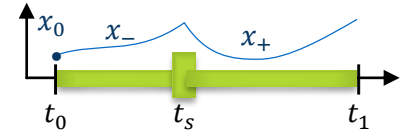
▪ Chain rule yields for the sensitivities:

$$G_{x_0}(t_1; t_0, x_0, p) = G_{x_0}(t_1; t_s, x_+(t_s), p) \cdot U \cdot G_{x_0}(t_s; t_0, x_0, p)$$

▪ Sensitivity update for model change and state jump Δ :

$$U = I + \frac{\partial \Delta}{\partial x_-} + \left(f_+ - f_- - \frac{\partial \Delta}{\partial t_s} - \frac{\partial \Delta}{\partial x_-} f_- \right) \cdot \frac{\frac{\partial \sigma_i}{\partial x_-}}{\frac{d\sigma_i}{dt_s}}$$

▪ IFDIFF • gives access to f_+ and f_- (by forced branching) and
• auto-generates required **differentiable** switching functions σ_i



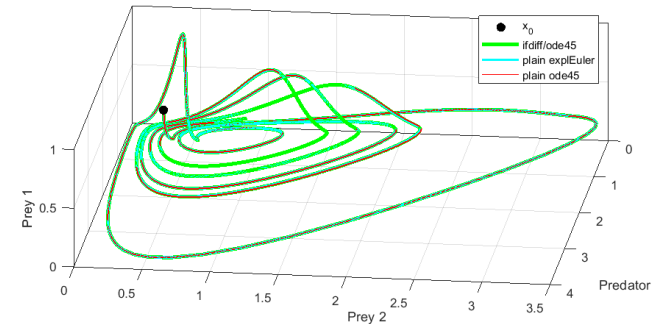
Note: t_s depends on x_0 and p
 $\Rightarrow$ implicit function theorem
on $\sigma(t_s, x, p) = 0$

$\Rightarrow$ correct sensitivities require
switching function derivatives

Summary

- Ignoring switches leads to poor integration and useless sensitivities – Unnoticed disasters ahead!
- IFDIFF is an automated Matlab toolkit for multi-dimensional ODEs with state-dependent switches providing ...
- ... accurate switching point detection
- ... accurate integration
- ... accurate forward sensitivities for states and parameters
- ... automatic Filippov sliding mode detection and handling
- ... and integrates seamlessly in Matlab's ode solver suite (ode23, ode45, ode15s, ...) delivering a `sol` structure compatible to Matlab's `deval` → **practical**

→ next lecture



View Project on



<https://andreassommer.github.io/ifdiff>



THANKS TO FUNDERS ...

VECTOR 
STIFTUNG



05M2022 - MORFAE

... AND COLLABORATORS

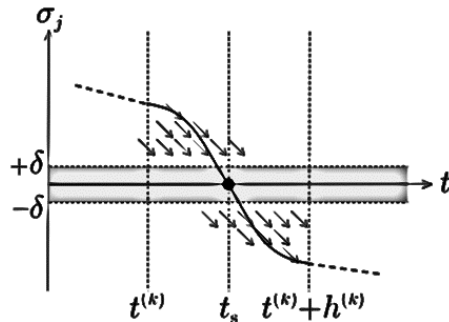
Kilian Folger
Lev Gromov
Pilar Keller
Tobias Reith
Ilona Slutsker
Michael Strik
Valentin von Trotha
Leonard Wirsching
Hans Georg Bock
Ekaterina Kostina

Switched ODEs: A Glimpse onto Solution Theory

- Initial value problem (IVP) with switches: $\dot{x}(t) = f(t, x(t), p, \text{sign } \sigma(t, x(t), p)) \quad x(t_0) = x_0(p)$
- Three types of switches (the good, the bad, and the ugly):

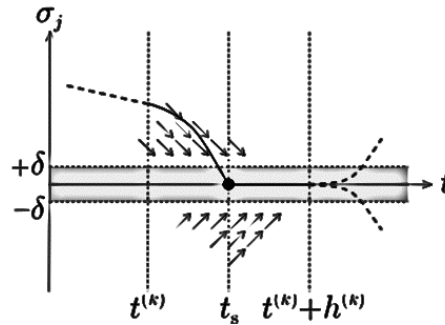
Consistent switch:
leaving $\sigma = 0$ in well-defined direction

$$\frac{d\sigma^-}{dt_s}(t_s) \cdot \frac{d\sigma^+}{dt_s}(t_s) > 0$$



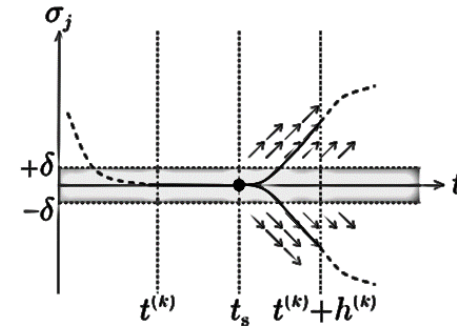
Inconsistent switch:
staying at $\sigma = 0$

$$\frac{d\sigma^-}{dt_s}(t_s) < 0 \quad \text{and} \quad \frac{d\sigma^+}{dt_s}(t_s) > 0$$



Bifurcation
leaving $\sigma = 0$ in both directions

$$\frac{d\sigma^-}{dt_s}(t_s) > 0 \quad \text{and} \quad \frac{d\sigma^+}{dt_s}(t_s) < 0$$



Illustrations: C. Kirches, A Numerical Method for Nonlinear Robust Optimal Control with Implicit Discontinuities and an Application to Powertrain Oscillations, 2006

Classical & CARATHEODORY solutions

CARATHEODORY & FILIPPOV solutions